

PRD: Craft Relay — Claude Desktop ↔ browser-hosted Craft Suite apps

Oct 9, 2026 · @Derek

Overview

Craft Relay lets Claude Desktop control the Craft Suite apps running in a browser tab on Railway, by passing MCP tool calls through a small relay service to a bridge script inside the page.

The problem. The deployed suite is static WebAssembly: the editing engine runs in the visitor's browser and nothing runs server-side. The desktop apps' MCP servers and control channels are loopback-only, so they cannot be reached from Railway. Claude can drive the web apps today only through browser automation, by calling in-page hooks such as `lightcraft.command(...)` and `effectcraft.execute(...)`.

Why a relay. A Claude Desktop custom connector connects from Anthropic's cloud to a public URL, so it cannot reach a tab on the user's own machine. The relay gives the connector a public MCP endpoint, and the tab connects out to the relay over a WebSocket. That outbound connection is what makes the tab reachable.

Status. Proposal only. Nothing here is built. Claims about Claude's connector behaviour and the apps' in-page APIs are marked for verification in the first spike (see Risks and open questions).

Goals and non-goals

The MVP succeeds when a person can add one custom connector in Claude Desktop, open a Craft Suite app, and edit in that app from a Claude chat.

Goals

- Expose each open Craft Suite tab as a set of MCP tools through one public, authenticated endpoint on Railway.
- Work with LightCraft and EffectCraft first, since both expose a documented in-page API; add the other four apps through adapters.
- Return images (a rendered frame or a screenshot) so Claude can check its own edits.

- Make pairing explicit: the user turns the bridge on in the page, sees when an agent is connected, and can cut it off at once.
- Deploy as an extra service in the existing Craft Suite Railway template.

Non-goals for the MVP

- Uploading or downloading files between Claude and the apps. Exports stay a manual step in the browser.
- Controlling the desktop apps. They already have local MCP servers.
- Multi-user accounts, per-user billing, or hosting other people’s sessions.
- OAuth sign-in for the connector. The MVP uses a static bearer token in a request header.
- Headless operation with no tab open. The engine lives in the page, so a tab must stay open.

Users and scenarios

The primary user is Derek, a creator who wants to show Claude Desktop operating the deployed suite on camera. A secondary user is any Craft Suite deployer who wants agent control without installing desktop binaries.

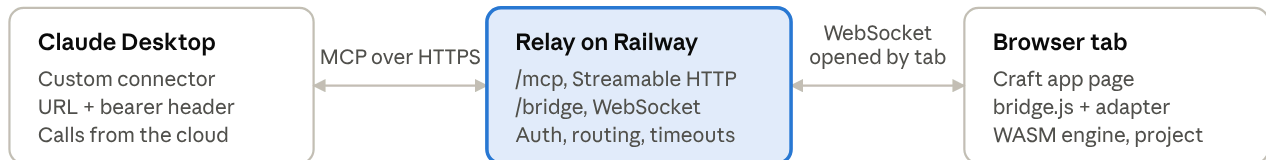
Scenario	What the user says to Claude	What should happen	App
Demo edit	"Make this photo warmer and brighter."	Claude lists controls, sets several, renders a preview, and reports the result.	LightCraft
Title card	"Make a 5-second title card that says Hello World."	Claude creates a composition and a text layer, and renders a frame to check it.	EffectCraft
Inspect	"What is open in each tab?"	Claude lists connected tabs with app, version and state.	All
Safe stop	The user clicks Disconnect in the page.	Further tool calls fail with a clear message within 2 seconds.	All

The on-camera scenario sets the quality bar: calls must feel live, and the page must visibly show that an agent is connected.

Architecture

Claude never talks to the tab directly: it calls the relay's public MCP endpoint, and the relay forwards each call down a WebSocket that the tab opened.

Claude reaches a browser tab only through the relay



The tab connects out, so no inbound port opens on the machine running it.

architecture · 3 components, 2 connections

Life of one tool call

1. The user clicks Connect agent in the app page and pastes the bridge token. The tab opens the WebSocket and sends a hello message with its app, version and tab id.
2. Claude calls a tool, for example `run_command`, on the connector URL with the bearer header.
3. The relay checks the token, picks the target tab, and sends it a JSON request with a unique id.
4. The bridge runs the adapter call inside the page and replies with the result, an error, or a downsized image.
5. The relay returns that as the MCP result. If no reply arrives within 20 seconds, it returns a timeout error instead.

Functional requirements

Three components are in scope: a bridge script in each app page, a relay service, and one adapter per app.

Bridge (runs in the page)

1. A "Connect agent" control in the app page starts the bridge. Nothing connects until the user clicks it.

2. The bridge opens a WebSocket to the relay, sends a hello message (app name, version from `versions.json`, a random tab id), and answers heartbeats.
3. It receives request messages, calls the app's in-page API through the adapter, and returns the JSON result or an error.
4. It shows a persistent indicator while connected, a Disconnect button, and a log of the last commands run.
5. It reconnects with backoff after a drop, and reports a lost GPU device (seen once in EffectCraft) as a distinct error rather than a timeout.
6. It downsizes images before sending them, because EffectCraft's `renderFrame` returned a PNG of about 4.9 million base64 characters for a 1280 by 720 frame.

Relay service (Node, on Railway)

1. Serves `POST/GET /mcp` over Streamable HTTP and `wss://.../bridge` for tabs, plus `/healthz`.
2. Rejects MCP requests that lack the configured bearer token, and rejects bridge sockets from origins outside an allowlist.
3. Keeps an in-memory registry of connected tabs and routes each tool call to one tab, with a per-call timeout (default 20 seconds).
4. Stores no user content. Logs record tool names, tab ids and timings, never parameters or images.

MCP tools

Tool	Purpose	Notes
<code>list_sessions</code>	List connected tabs: session id, app, version, last seen	Default target is the most recently active tab
<code>list_commands</code>	List an app's commands with parameter hints; optional text filter	LightCraft has no list command, so its adapter ships a static catalog
<code>run_command</code>	Run one command with JSON params and return its result	Applies the denylist from the security section
<code>inspect</code>	Return app state (active comp, selection, library info)	Maps to <code>ui.inspect</code> or <code>library.info</code>
<code>get_image</code>	Return a preview as MCP image content, capped at 1 MB	Frame render where the app supports it; screenshot otherwise

Tool	Purpose	Notes
<code>run_batch</code>	Run up to 20 commands in order and stop on the first error	Cuts round trips for multi-step edits (should have, not must)

App adapters

App	In-page API found	Adapter status
LightCraft	<code>lightcraft.command(name, jsonString);</code> confirmed <code>library.info</code> , <code>develop.controls</code> , <code>develop.get</code> , <code>develop.set {control, value}</code>	Ready to build
EffectCraft	<code>effectcraft.execute, commands, inspect,</code> <code>screenshot, renderFrame</code> ; 665 commands	Ready to build
FilmCraft	Docs describe <code>window.filmcraft.execute(...)</code>	Confirm in spike
PhotoCraft, PdfCraft, VectorCraft	Not checked	Discover in spike

Security and privacy

The relay is a public endpoint that can run commands inside a person's browser app, so authentication and a visible kill switch are release blockers, not polish.

Threat	Requirement
Stranger calls <code>/mcp</code>	Require a bearer token (at least 32 random bytes, generated at deploy, compared in constant time). Claude stores it as a connector request header. Rotating it means changing one environment variable and the header.

Threat	Requirement
Rogue site or tab opens a bridge socket	Check the <code>Origin</code> header against an allowlist, and require a second secret (the bridge token) that the user pastes into the Connect dialog. Keep it in memory or <code>sessionStorage</code> , never in the page source.
Agent runs a destructive command	Ship a default denylist (quit, close project, delete library, bulk overwrite) and a "read-only" toggle in the page. Denied calls return a clear error. The exact command IDs are collected in the spike.
Arbitrary code execution in the tab	Expose no generic JavaScript or <code>eval</code> tool. Adapters call only the documented in-page API.
Prompt injection through app content	Treat text returned from the app (layer names, photo captions, PDF text) as data in tool results. Keep tools narrow so a hostile caption cannot reach anything beyond the allowed commands.
Leaking user media	Send images only when <code>get_image</code> is called, cap them at 1 MB, and keep nothing on the relay. Use TLS only. Never log parameters or image bytes.
Abuse or runaway loops	Rate limit to 10 calls per second per token, cap messages at 2 MB, allow at most 5 tabs per relay, and time out calls at 20 seconds.

Privacy statement for the page. "While connected, an agent can run commands in this tab and request previews of what you are editing. Nothing is stored on the relay. Disconnect at any time."

Railway's public domain gives TLS. The relay needs no database.

Non-functional requirements and success metrics

The targets below are proposed starting points, to be tuned once the spike measures real latency.

Area	Target
Command latency	Median under 500 ms for a simple command (for example one <code>develop.set</code>), measured from Claude's tool call to its result
Image preview	Under 3 seconds for a 1024 px preview, and under 1 MB
Reconnect	Bridge restores its socket within 10 seconds of a network drop, and the relay drops a dead tab within 30 seconds
Availability	One Railway replica is enough for the MVP. The registry is in memory, so a relay restart disconnects all tabs and they reconnect on their own
Footprint	Under 256 MB of memory, with no database or volume
Browser support	Chrome and Edge, matching the suite's own WebGPU recommendation
Background tabs	Connection survives a hidden tab for at least 10 minutes. The relay sends WebSocket pings so throttled timers do not cause false disconnects

Success for the MVP

1. The two demo scenarios (LightCraft edit, EffectCraft title card) run end to end from a Claude Desktop chat on camera, with no manual retries.
2. A new deployer goes from the Railway template to a working connector in under 10 minutes using the README alone.
3. No unauthenticated request ever reaches a tab, checked by tests for missing token, wrong token and wrong origin.

Delivery plan

Build the relay as a separate service in the Craft Suite Railway template, with the bridge shipped as a static script beside the apps. Day counts below are rough estimates for one developer, not commitments.

Repository layout (proposed)

- `server/` : Node and TypeScript relay, using the official MCP TypeScript SDK for the Streamable HTTP endpoint and the `ws` package for bridge sockets.
- `bridge/` : `bridge.js` plus one adapter file per app, served from the suite's static files.
- `Dockerfile` and a Railway config for the relay, and a README with the connector setup steps.

Configuration. The relay reads `MCP_TOKEN`, `BRIDGE_TOKEN`, `ALLOWED_ORIGINS`, `PORT` and `CALL_TIMEOUT_MS`. The suite pages read the relay's public address from a small config file generated at deploy time. In the template, the relay's domain is passed to the suite service as a variable.

Milestones, each with an exit gate

1. **Spike (about 1 day).** Prove a hand-written MCP server over Streamable HTTP can be added as a Claude Desktop custom connector with a bearer header, and that a tool call reaches a tab and returns. Gate: all open questions below are answered.
2. **LightCraft end to end (about 3 days).** Relay, bridge and the LightCraft adapter, with `list_sessions`, `run_command`, `inspect` and `get_image`. Gate: the "warmer and brighter" demo works from Claude Desktop.
3. **EffectCraft and hardening (about 3 days).** EffectCraft adapter, denylist, read-only mode, rate limits, reconnect handling and the security tests. Gate: the "Hello World" title card demo works, and every row of the security table has a test.
4. **Remaining apps and template (about 3 days).** FilmCraft, PhotoCraft, PdfCraft and VectorCraft adapters as their APIs allow, the Railway template service, and the README. Gate: a clean deploy from the template connects in under 10 minutes.
5. **Demo and release.** Record the video segment and tag a first release.

Risks, open questions and alternatives

The biggest risk is that the project depends on app APIs that are days or weeks old, so adapters will need version pinning and some rework.

Open questions for the spike

Question	Why it matters	How to check
Does a Claude Desktop custom connector accept a Streamable HTTP server that authenticates with a static bearer request header?	The whole design assumes it. Claude's help page lists request headers and a "No sign in" mode but does not name the transports.	Stand up a minimal MCP server on Railway and add it as a connector
Does Claude use image content returned by a tool?	<code>get_image</code> is how Claude checks its edits.	Return a small PNG from the test server
Do FilmCraft, PhotoCraft, PdfCraft and VectorCraft expose in-page APIs like LightCraft and EffectCraft?	Decides how many apps ship in the MVP. FilmCraft's docs mention <code>window.filmcraft</code> ; the others were not checked.	Read each page's source and docs
Does the suite's cross-origin isolation (COOP and COEP headers) block a WebSocket to the relay's domain?	A blocked socket breaks the bridge. WebSockets are not expected to be affected.	Open a socket from a deployed app page
Does a hidden tab keep its socket and answer within the timeout?	Demos may run with the app tab in the background.	Hide a tab for 10 minutes while pinging it
Can previews be downsized in the page reliably?	A GPU device was lost once in EffectCraft, and a frame returned about 4.9 million base64 characters.	Resize in a canvas, and test after a GPU loss
Which commands are destructive in each app?	Needed for the denylist.	Review each app's command catalog

Other risks

- App versions move fast (the suite shows v0.4 to v0.7). Pin adapters to the version in `versions.json` and fail with a clear message on a mismatch.
- The tab must stay open. A closed tab means no tools, which the relay reports plainly.
- One in-memory relay replica means a restart drops all sessions. Acceptable for the MVP.

Alternatives considered

Option	Status	Limit
Claude drives the page with browser automation	Works today (confirmed on LightCraft)	Needs a Claude browser on the same machine and cannot be used from a plain chat
Desktop CLI as a local MCP server	Documented in each repo	A separate install, not the Railway deployment
Headless CLI on Railway behind a stdio-to-HTTP gateway	Possible	No page UI, no access to the user's files, and no documented inbound authentication on the gateway